Week 5: Population Projections Lab

SOC6708 ADA

Liangqi (Cecilia) Shen

For this lab, we are going to pick two countries from the WPP data (one high fertility, one low fertility) and do some projections and calculations.

Load in data

```
library(tidyverse)
```

Loading in the data. For this lab we will need fertility rates, populations by age, and the Lx columns of the life table.

```
df <- read_csv("../desktop/WPP2024_Fertility_by_Age5.csv")
df <- df |> select(Location, Time, ASFR, AgeGrp)
dl <- read_csv("../desktop/WPP2024_Life_Table_Abridged_Medium_1950-2023.csv")
dl <- dl |> filter(Variant=="Medium", Sex=="Female")|> select(Location, Time, AgeGrp,Lx)
d_female <- read_csv("../desktop/wpp_pop_clean.csv") |> filter(sex=="Female")
d_female <- d_female |>
  pivot_longer(x0:x100) |>
  mutate(age = as.numeric(str_remove(name, "x")))
# create age group (need to sum over later)
breaks <- c(seq(0,100, 5), Inf)
labels <- c("0-4", "5-9", "10-14", "15-19", "20-24", "25-29", "30-34", "35-39",
            "40-44", "45-49", "50-54", "55-59", "60-64", "65-69", "70-74",
            "75-79", "80-84", "85-89",
            "90-94", "95-99", "100+")

d_female$age_group <-  cut(d_female$age, breaks = breaks, labels = labels, right = FALSE)
```

Population projection

1. pick two countries, 1 with high fertility and 1 with low fertility. Here are the options:

```
#unique(df$Location)
# Cecilia to do japan and niger
```

2. Calculate the NRR for each of the countries in 2020. Confirm that these are one high and one low, based on NRR values.

```
df |>
  filter(Location == "Japan"|Location == "Niger", Time == 2020) |>
  left_join(dl) |>
  mutate(prod = ASFR/1000*Lx/10^5) |>
  group_by(Location) |>
  summarize(NRR = sum(prod)*0.4886)
```

```
# A tibble: 2 x 2
  Location   NRR
  <chr>    <dbl>
1 Japan    0.630
2 Niger    2.60
```

Remember: discrete version of NRR is

$$NRR = \sum_x {}_nL_x \cdot {}_nF_x \cdot f_{fab}$$

and the fraction female at birth is something like 0.4886.

Leslie matrix

3. Create a Leslie matrix for each of your countries based on 2020 rates. The function below will help. This is pretty ugly so if someone want to rewrite it feel free :)

Note: Fx only covers 10-50, but it needs to be 0 for all other age groups (probably easiest to `left_join` `df` to `dl` then replace the `NA` values with 0s). Note 2: From the WPP data we have a split first age group, but want to convert this into 0-5. Just sum the Lx values to get ${}_5L0$

```r
leslie <- function(nLx,
                   nFx,
                   n_age_groups=21, # based on 0 to 100+
                   ffab = 0.4886){
  L = matrix(0, nrow = n_age_groups, ncol = n_age_groups)
  L[1,] = ffab * nLx[1]*(nFx[1:n_age_groups]+nFx[2:(n_age_groups+1)]*nLx[2:(n_age_groups+1)]/
  L[1,ncol(L)] <- 0
  diag(L[2:n_age_groups,1:(n_age_groups-1)]) = nLx[2:n_age_groups] / nLx[1:(n_age_groups-1)]
  return(L)
}
```

```r
d_jap <- dl |>
  filter(Location == "Japan", Time == 2020) |>
  left_join(df) |>
  mutate(ASFR = ifelse(is.na(ASFR), 0, ASFR)) |>
  mutate(age_group = ifelse(AgeGrp=="0"|AgeGrp=="1-4", "0-5", AgeGrp)) |>
  group_by(age_group) |>
  summarize(Lx = sum(Lx/10^5), Fx = sum(ASFR/1000)) |>
  mutate(age = as.numeric(str_remove(age_group, "-.*"))) |>
  mutate(age = ifelse(is.na(age), 100, age)) |>
  arrange(age)

Fx_jap <- d_jap$Fx
Lx_jap <- d_jap$Lx

A_jap <- leslie(nLx = Lx_jap, nFx = Fx_jap)

A_jap
```

```
           [,1]         [,2]         [,3]       [,4]      [,5]      [,6]
 [1,] 0.0000000 2.437645e-05 0.003074334 0.03056293 0.1171996 0.2052752
 [2,] 0.9995607 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
 [3,] 0.0000000 9.996910e-01 0.000000000 0.00000000 0.0000000 0.0000000
 [4,] 0.0000000 0.000000e+00 0.999441182 0.00000000 0.0000000 0.0000000
 [5,] 0.0000000 0.000000e+00 0.000000000 0.99898355 0.0000000 0.0000000
 [6,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.9987558 0.0000000
 [7,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.9986389
 [8,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
 [9,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[10,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[11,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[12,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
```

```
[13,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[14,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[15,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[16,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[17,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[18,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[19,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[20,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
[21,] 0.0000000 0.000000e+00 0.000000000 0.00000000 0.0000000 0.0000000
          [,7]      [,8]       [,9]         [,10]     [,11]     [,12]     [,13]
 [1,] 0.1820259 0.0800627 0.01419379 0.0005839963 0.0000000 0.0000000 0.0000000
 [2,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [3,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [4,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [5,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [6,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [7,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [8,] 0.9981399 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
 [9,] 0.0000000 0.9971593 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[10,] 0.0000000 0.0000000 0.99560948 0.0000000000 0.0000000 0.0000000 0.0000000
[11,] 0.0000000 0.0000000 0.00000000 0.9930670285 0.0000000 0.0000000 0.0000000
[12,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.9900251 0.0000000 0.0000000
[13,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.9861334 0.0000000
[14,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.9794245
[15,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[16,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[17,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[18,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[19,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[20,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
[21,] 0.0000000 0.0000000 0.00000000 0.0000000000 0.0000000 0.0000000 0.0000000
          [,14]     [,15]    [,16]     [,17]     [,18]     [,19]     [,20]
 [1,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [2,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [3,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [4,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [5,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [6,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [7,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [8,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
 [9,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[10,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[11,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
```

```
[12,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[13,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[14,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[15,] 0.9674156 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[16,] 0.0000000 0.9458685 0.000000 0.0000000 0.0000000 0.0000000 0.0000000
[17,] 0.0000000 0.0000000 0.901207 0.0000000 0.0000000 0.0000000 0.0000000
[18,] 0.0000000 0.0000000 0.000000 0.8100966 0.0000000 0.0000000 0.0000000
[19,] 0.0000000 0.0000000 0.000000 0.0000000 0.6464464 0.0000000 0.0000000
[20,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.4295751 0.0000000
[21,] 0.0000000 0.0000000 0.000000 0.0000000 0.0000000 0.0000000 0.2454475
      [,21]
 [1,]     0
 [2,]     0
 [3,]     0
 [4,]     0
 [5,]     0
 [6,]     0
 [7,]     0
 [8,]     0
 [9,]     0
[10,]     0
[11,]     0
[12,]     0
[13,]     0
[14,]     0
[15,]     0
[16,]     0
[17,]     0
[18,]     0
[19,]     0
[20,]     0
[21,]     0
```

```r
d_nig <- dl |>
  filter(Location == "Niger", Time == 2020) |>
  left_join(df) |>
  mutate(ASFR = ifelse(is.na(ASFR), 0, ASFR)) |>
  mutate(age_group = ifelse(AgeGrp=="0"|AgeGrp=="1-4", "0-5", AgeGrp)) |>
  group_by(age_group) |>
  summarize(Lx = sum(Lx/10^5), Fx = sum(ASFR/1000)) |>
  mutate(age = as.numeric(str_remove(age_group, "-.*"))) |>
  mutate(age = ifelse(is.na(age), 100, age)) |>
```

```
  arrange(age)

Fx_nig <- d_nig$Fx
Lx_nig <- d_nig$Lx

A_nig <- leslie(nLx = Lx_nig, nFx = Fx_nig)

A_nig
```

```
           [,1]        [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
 [1,] 0.0000000 0.004732836 0.1726270 0.4718290 0.6156429 0.5881281 0.4717217
 [2,] 0.9623851 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
 [3,] 0.0000000 0.992578346 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
 [4,] 0.0000000 0.000000000 0.9917331 0.0000000 0.0000000 0.0000000 0.0000000
 [5,] 0.0000000 0.000000000 0.0000000 0.9882432 0.0000000 0.0000000 0.0000000
 [6,] 0.0000000 0.000000000 0.0000000 0.0000000 0.9865664 0.0000000 0.0000000
 [7,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.9858037 0.0000000
 [8,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.9836768
 [9,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[10,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[11,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[12,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[13,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[14,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[15,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[16,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[17,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[18,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[19,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[20,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
[21,] 0.0000000 0.000000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
           [,8]      [,9]       [,10]        [,11]     [,12]     [,13]     [,14]
 [1,] 0.2981611 0.1449323 0.04490315 0.002742035 0.0000000 0.0000000 0.0000000
 [2,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [3,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [4,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [5,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [6,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [7,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [8,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
 [9,] 0.9796059 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
[10,] 0.0000000 0.9748566 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
```

```
[11,] 0.0000000 0.0000000 0.96618534 0.000000000 0.0000000 0.0000000 0.0000000
[12,] 0.0000000 0.0000000 0.00000000 0.951856351 0.0000000 0.0000000 0.0000000
[13,] 0.0000000 0.0000000 0.00000000 0.000000000 0.9263943 0.0000000 0.0000000
[14,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.8799932 0.0000000
[15,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.8045284
[16,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
[17,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
[18,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
[19,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
[20,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
[21,] 0.0000000 0.0000000 0.00000000 0.000000000 0.0000000 0.0000000 0.0000000
          [,15]     [,16]     [,17]     [,18]      [,19]     [,20] [,21]
 [1,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [2,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [3,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [4,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [5,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [6,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [7,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [8,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
 [9,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[10,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[11,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[12,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[13,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[14,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[15,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[16,] 0.6899213 0.0000000 0.0000000 0.0000000 0.00000000 0.0000000     0
[17,] 0.0000000 0.5367206 0.0000000 0.0000000 0.00000000 0.0000000     0
[18,] 0.0000000 0.0000000 0.3523425 0.0000000 0.00000000 0.0000000     0
[19,] 0.0000000 0.0000000 0.0000000 0.1785819 0.00000000 0.0000000     0
[20,] 0.0000000 0.0000000 0.0000000 0.0000000 0.07160207 0.0000000     0
[21,] 0.0000000 0.0000000 0.0000000 0.0000000 0.00000000 0.0258138     0
```

4. Confirm that the NRRs from the Leslie matrices are the same as your NRRs above. The formula is:

$$NRR = \sum A_{1,j(x)} \frac{{}_nL_x}{{}_nL_0}$$

where A is the Leslie matrix. I think the easiest way to do this in practice is `A[1,]%*%cumprod(c(1,diag(A[-1,])))`.

```
A_jap[1,]%*%cumprod(c(1,diag(A_jap[-1,])))
```

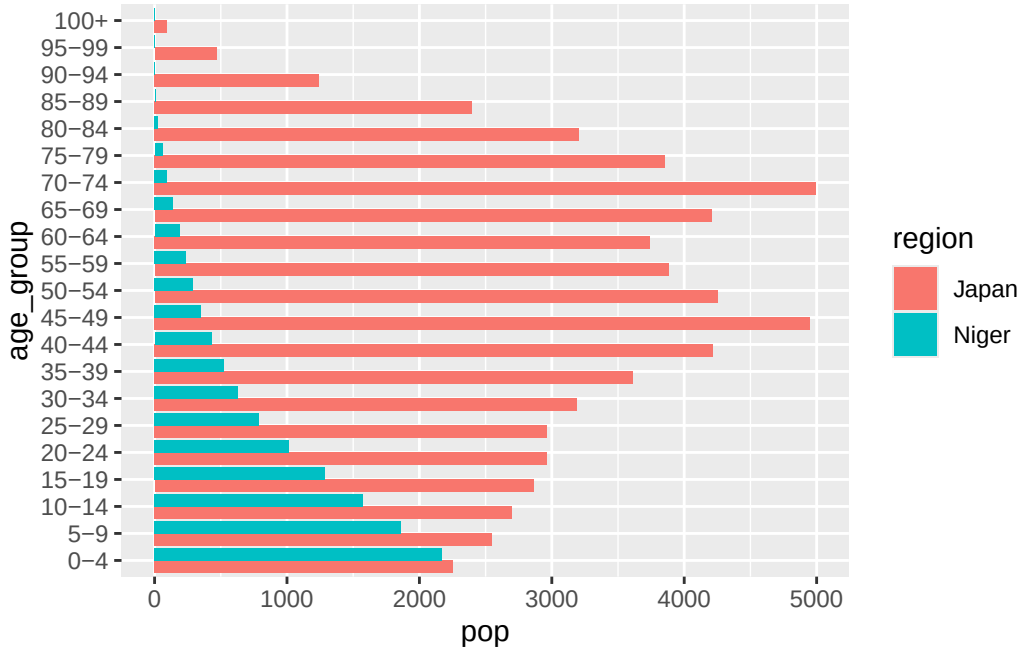
```
           [,1]
[1,] 0.6303753
```

```
A_nig[1,]%*%cumprod(c(1,diag(A_nig[-1,])))
```

```
          [,1]
[1,] 2.600421
```

5. Get the 2020 populations for your two chosen countries. What does the age distribution look like (plot half an age pyramid). Given the NRRs what do expect the age pyramids ending up to look like?

```
dpop <- d_female |>
  filter(region == "Japan"|region == "Niger", year == 2020) |>
  group_by(region, age_group) |>
  summarize(pop = sum(value))

dpop |>
  ggplot(aes(age_group, pop, fill = region)) +
  geom_bar(stat = "identity", position = "dodge")+
  coord_flip()
```



6. Project each population forward 200 years (i.e. 200/5 = 40 projection steps). For each country, you'll want to save every population for each age group and projection step in a matrix, and put the first column as your starting population. So something like this:

```
# for country 1
n <- 5
age_groups <- seq(0, 100, by = n)
n_age_groups <-  length(age_groups)
n_projections <- 200/n
initial_pop <- dpop |> filter(region=="Japan") |> select(pop) |> pull()
# define population matrix K
K <- matrix(0, nrow = n_age_groups, ncol = n_projections+1)
K[,1] <- initial_pop

# do the projection!
for(i in 2:(n_projections+1)){
 this_projection <- A_jap%*%K[,i-1]
 K[,i] <- this_projection
}

# for country 2
K_nig <- matrix(0, nrow = n_age_groups, ncol = n_projections+1)
K_nig[,1] <- dpop |> filter(region=="Niger") |> select(pop) |> pull()

# do the projection!
for(i in 2:(n_projections+1)){
 this_projection <- A_nig%*%K_nig[,i-1]
 K_nig[,i] <- this_projection
}
```

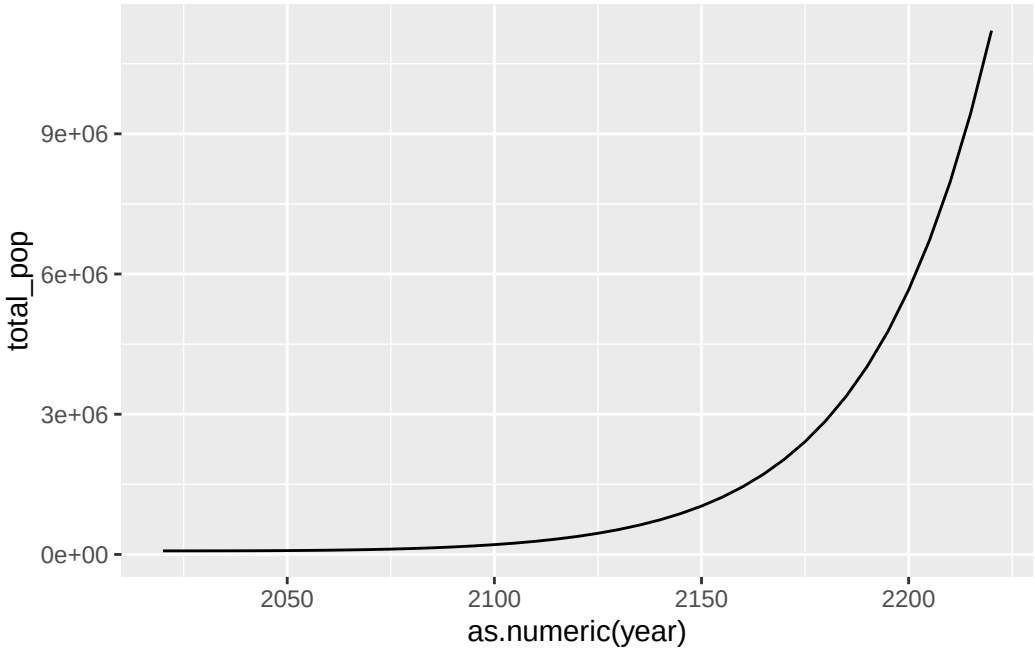
For the next bits, you probably want to take these two matrices and convert them back into a tibble for plotting. Something like this:

```
# for country 1
Kdf <- as_tibble(K)
colnames(Kdf) <- seq(from = 2020, to = (2020+n_projections*n), by = 5)
Kdf <- cbind(age = seq(from = 0, to = 100, by = 5), Kdf)

# get in long format and then add proportion of population in each age group

Kdf_long <- Kdf |>
  pivot_longer(-age, names_to = "year", values_to = "pop") |>
```

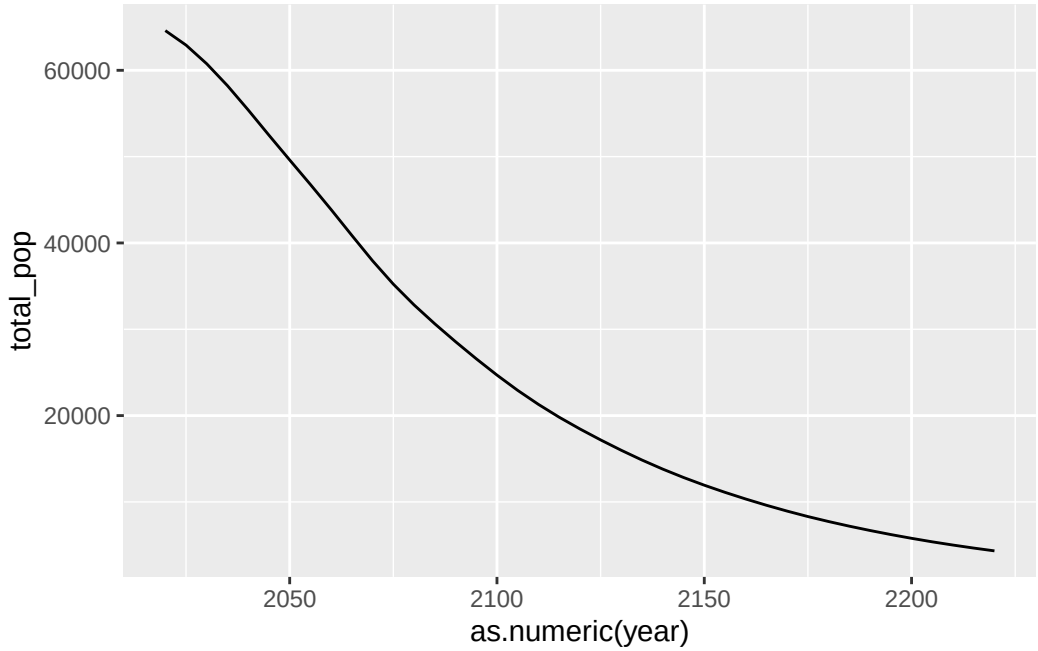
```
  group_by(year) |>
  mutate(prop = pop/sum(pop))

# do the same for country 2
Kdf_nig <- as_tibble(K_nig)
colnames(Kdf_nig) <- seq(from = 2020, to = (2020+n_projections*n), by = 5)
Kdf_nig <- cbind(age = seq(from = 0, to = 100, by = 5), Kdf_nig)

# get in long format and then add proportion of population in each age group

Kdf_long_nig <- Kdf_nig |>
  pivot_longer(-age, names_to = "year", values_to = "pop") |>
  group_by(year) |>
  mutate(prop = pop/sum(pop))

# then join them together

# add country label
Kdf_long <- Kdf_long |>
  mutate(country = "Japan")

Kdf_long_nig <- Kdf_long_nig |>
  mutate(country = "Niger")

# combine both countries
Kdf_all <- bind_rows(Kdf_long, Kdf_long_nig)

# optional: reorder columns
Kdf_all <- Kdf_all |>
  select(country, year, age, pop, prop)
```

7a. Plot the total population size over time

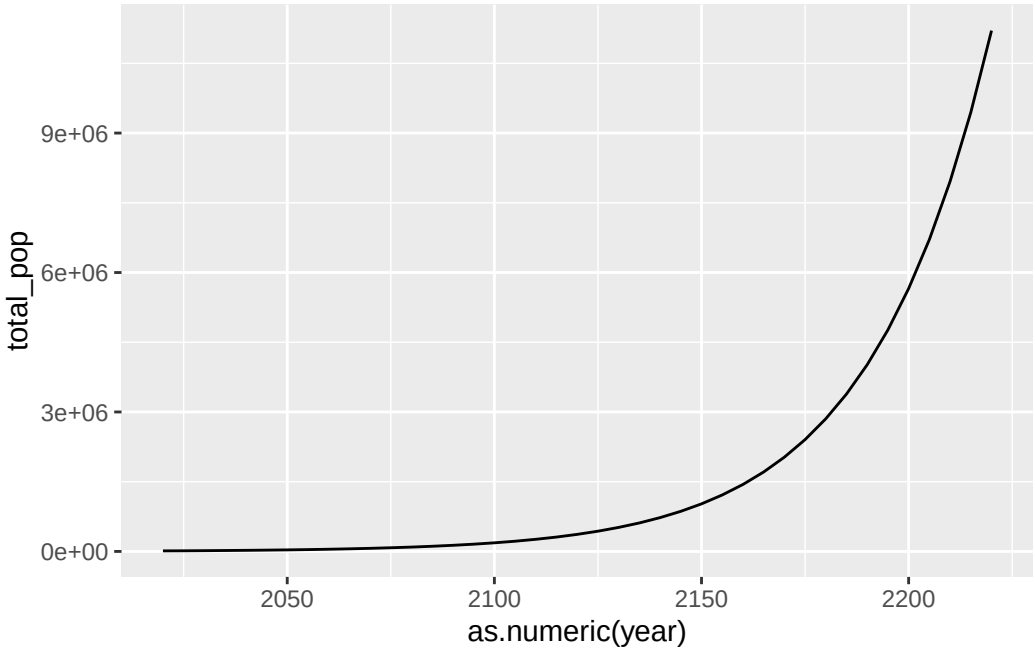
```
Kdf_all |>
  group_by(year) |>
  summarize(total_pop = sum(pop)) |>
  ggplot(aes(as.numeric(year), total_pop)) +
  geom_line()
```




```
Kdf_long |>
  group_by(year) |>
  summarize(total_pop = sum(pop)) |>
  ggplot(aes(as.numeric(year), total_pop)) +
  geom_line()
```

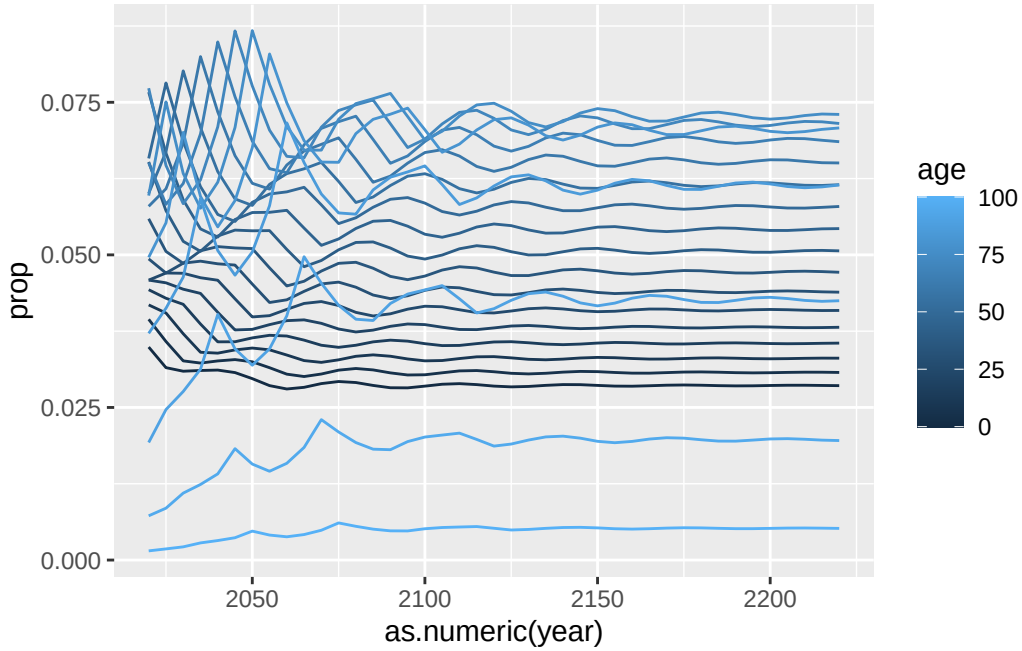



```
Kdf_long_nig |>
  group_by(year) |>
  summarize(total_pop = sum(pop)) |>
  ggplot(aes(as.numeric(year), total_pop)) +
  geom_line()
```



7b. Plot the proportion in each age group over time

```
Kdf_long |>
  ggplot(aes(as.numeric(year), prop, color = age, group = age)) +
  geom_line()
```




```
Kdf_long_nig |>
  ggplot(aes(as.numeric(year), prop, color = age, group = age)) +
  geom_line()
```